

# AI Models in Autonomous Game Development

## A Comparative Study on Efficiency, Quality, and Societal Impact

### Front Matter

#### Abstract

The rapid advancement of Large Language Models (LLMs) has precipitated a paradigm shift in software engineering, particularly in the domain of autonomous code generation. This report investigates the efficacy, efficiency, and broader impacts of utilizing diverse AI models for autonomous game development. Through a controlled experiment, five distinct AI models (Claude Haiku 4.5, Grok Code Fast 1, GPT-5.3-Codex, Gemini 3.1 Pro, and Raptor mini) were tasked with developing a fully functional, visually complete Chess game using the PyGame library, based on a rigorous Game Design Document (GDD). The study evaluates the models based on prompt requirements, token consumption, model turns, tool calls, and overall output quality (mechanical accuracy and aesthetic presentation).

Beyond technical performance, this report significantly widens the scope of traditional software benchmarking by incorporating an in-depth analysis of the environmental and societal impacts of AI-driven development. By examining the thermodynamics of LLM inference, calculating Power Usage Effectiveness (PUE), and estimating the energy and water consumption per token, this study addresses the growing ecological footprint of generative AI. Furthermore, it explores the societal implications, including the democratization of game development, potential disruptions to the labor market, and copyright considerations. The findings provide comprehensive guidelines for selecting appropriate AI models, balancing developmental efficiency with environmental sustainability and ethical software practices.

#### List of Tables and Figures

- *Table 1.1:* Overview of Tested AI Models and Target Application
- *Table 4.1:* Comprehensive Model Performance Summary (Prompts, Tokens, Errors)
- *Table 4.2:* S-Tier, A-Tier, and B-Tier Subjective Rankings
- *Table 5.1:* Estimated Energy Consumption, Carbon, and Water Footprint per Model
- *Figure 1:* Token-to-Quality Ratio Across Evaluated Models
- *Figure 2:* UI and Visual Output Comparisons (Selected Models)

## [AI Models in Autonomous Game Development](#)

### [A Comparative Study on Efficiency, Quality, and Societal Impact](#)

#### [Front Matter](#)

##### [Abstract](#)

##### [List of Tables and Figures](#)

#### [Chapter 1: Introduction](#)

##### [1.1 Background and Motivation](#)

##### [1.2 Problem Statement & Research Objectives](#)

##### [1.3 Scope of the Study](#)

##### [1.4 Document Structure](#)

#### [Chapter 2: Literature Review](#)

##### [2.1 The Evolution of AI in Game Development](#)

##### [2.2 Large Language Models \(LLMs\) in Autonomous Code Generation](#)

##### [2.3 Challenges in LLM-driven Game Generation](#)

##### [2.4 The Environmental Footprint of AI: Measuring Energy Consumption per Token](#)

#### [Chapter 3: Methodology](#)

##### [3.1 Experiment Design and Objective](#)

##### [3.2 The Game Design Document \(GDD\) Baseline](#)

##### [3.3 Selected AI Models](#)

##### [3.4 Evaluation Metrics](#)

##### [3.5 Energy Consumption Estimation Methodology](#)

#### [Chapter 4: Results & Model Performance Analysis](#)

##### [4.1 Overall Performance Summary](#)

##### [4.2 S-Tier Analysis: GPT-5.3-Codex](#)

##### [4.3 A-Tier Analysis: Gemini 3.1 Pro and Claude Haiku 4.5](#)

##### [4.4 B-Tier Analysis: Grok Code Fast 1 and Raptor mini](#)

##### [4.5 Token-to-Quality Ratio & Efficiency Findings](#)

#### [Chapter 5: Environmental & Economic Impact](#)

##### [5.1 The Thermodynamics of Token Generation](#)

##### [5.2 Comparative Carbon and Water Footprint](#)

##### [5.3 The Jevons Paradox in AI Development](#)

##### [5.4 Cost-Performance Ratios vs. Human Labor](#)

##### [5.5 Sustainable AI Practices for Game Studios](#)

#### [Chapter 6: Societal Aspects and Impacts](#)

##### [6.1 Democratization of Game Development](#)

##### [6.2 The Future Role of Human Developers and Job Market Implications](#)

##### [6.3 Ethical Considerations, Copyright, and Plagiarism in AI-generated Code](#)

##### [6.4 Impact on the Indie Game Industry and Storefront Curation](#)

##### [6.5 Educational and Academic Implications](#)

#### [Chapter 7: Conclusions & Future Work](#)

##### [7.1 Summary of Key Findings](#)

##### [7.2 Recommendations for Game Developers](#)

##### [7.3 Limitations of the Study](#)

##### [7.4 Future Research Directions](#)

#### [References](#)

# Chapter 1: Introduction

## 1.1 Background and Motivation

The intersection of Artificial Intelligence (AI) and software engineering has reached a critical inflection point. Historically, AI in video games was strictly confined to controlling Non-Player Characters (NPCs) or managing procedural content generation (PCG) [1]. However, the advent of sophisticated Large Language Models (LLMs) has expanded AI's role from in-game actor to external developer. Modern LLMs are now capable of autonomous code generation, debugging, and architectural planning, effectively acting as junior developers or co-pilots.

Despite these advancements, game development remains a uniquely complex challenge for AI. Unlike standard web development or data processing scripts, game development requires a synthesis of logical state management, real-time rendering loops, event handling, and aesthetic visual integration. The motivation for this study arises from the need to empirically evaluate how well current state-of-the-art models can handle these multifaceted requirements. Furthermore, as part of the Information Society Technologies research initiative, there is an urgent need to contextualize this technological leap within its broader environmental and societal consequences. The computational cost of querying massive AI models is non-trivial, and as game studios begin integrating these tools into their pipelines, understanding the balance between code generation efficiency and energy consumption becomes imperative [2].

## 1.2 Problem Statement & Research Objectives

While AI models frequently demonstrate the ability to generate simple code snippets, their capacity to autonomously develop a complete, multi-layered interactive application from a single, extensive specification remains inconsistently documented. Developers lack empirical benchmarks to determine which models offer the best balance of quality, token efficiency, and computational cost.

Therefore, the primary problem this study addresses is the quantification of AI model efficiency and capability in autonomous game development. The core research objectives are:

1. **Performance Evaluation:** To measure the exact number of prompts, tokens, and model turns required by five distinct AI models to successfully implement a complete Chess game based on a stringent Game Design Document (GDD).
2. **Quality Assessment:** To subjectively and objectively evaluate the final outputs based on mechanical compliance to chess rules and visual/UI aesthetic quality.
3. **Environmental Analysis:** To rate model energy efficiency by analyzing the thermodynamics of token generation, data center overhead, and calculating the corresponding energy (Joules/mWh) and water consumption per project.

4. **Societal Impact Assessment:** To evaluate the practical suitability of these models for game development workflows and analyze the resulting socioeconomic implications, including job market shifts and copyright ethics.

## 1.3 Scope of the Study

To ensure a controlled, fair, and reproducible experiment, the scope of the technical evaluation is strictly bounded. The baseline task is the creation of a local, two-player Chess game using the Python pygame library. The implementation must adhere strictly to the provided GDD.md, which details setup, board dimensions (896x576p), asset integration, UI panels, and complex mechanics (e.g., castling, en passant, promotion, and check/checkmate validation).

The evaluated models include:

- **Claude Haiku 4.5**
- **Grok Code Fast 1**
- **GPT-5.3-Codex**
- **Gemini 3.1 Pro**
- **Raptor mini**

**Widening the Scope:** To fulfill the requirements of modern technological assessments, the scope of this report extends far beyond technical benchmarking.

- **Environmental Scope:** The study will analyze the cost-performance ratios and energy consumption associated with the millions of tokens generated during this experiment (e.g., the 12,000,000+ tokens consumed by Raptor mini vs. the 900,157 tokens used by GPT-5.3-Codex).
- **Societal Scope:** The report investigates the democratization of indie game development, the ethical dilemmas of AI copyright attribution, and the projected impacts on human software engineers.

## 1.4 Document Structure

This document is structured to guide the reader from theoretical background through empirical data and into broader societal analysis:

- **Chapter 2 (Literature Review)** explores previous research on AI in game development, LLM coding capabilities, and current studies on AI energy consumption.
- **Chapter 3 (Methodology)** details the experimental setup, the GDD specifications, the prompting strategy, and the metrics used for evaluation.
- **Chapter 4 (Results & Analysis)** provides a granular breakdown of each model's performance, classifying them into S, A, and B tiers based on empirical data and output quality.
- **Chapter 5 (Environmental & Economic Impact)** analyzes the thermodynamics of token generation to estimate carbon and water footprints, while evaluating cost-effectiveness.

- **Chapter 6 (Societal Aspects and Impacts)** discusses the broader implications of AI autonomous developers on the indie game industry, copyright law, and the educational sector.
- **Chapter 7 (Conclusions & Future Work)** summarizes the findings, offers recommendations for game developers, and outlines limitations and areas for further study.

## Chapter 2: Literature Review

### 2.1 The Evolution of AI in Game Development

Historically, Artificial Intelligence in video games has been defined by its utility inside the game environment rather than as a tool for creating the game itself. Early milestones, such as the computerized game of *Nim* (1942) and the pattern-based enemies in *Pac-Man* (1980), established a baseline for opponent behavior utilizing finite-state machines and behavior trees [1]. Over the decades, game AI evolved to tackle pathfinding, resource management, and complex decision-making in real-time strategy environments.

Simultaneously, Procedural Content Generation (PCG) emerged as a method to circumvent the limitations of hardware memory and human authoring time. Landmark titles like *Elite* (1984) and later *No Man's Sky* utilized algorithmic generation to build expansive, near-infinite worlds from mathematical seeds [3]. Traditional PCG, however, relies heavily on deterministic rules, grammars, and noise functions authored by human engineers. Recently, there has been a significant shift toward integrating machine learning into PCG, creating dynamic environments and quests that adapt to player behavior [3]. Today, the application of Large Language Models (LLMs) represents the next evolutionary leap: moving AI from a sub-system *within* the game to the *architect* of the game.

### 2.2 Large Language Models (LLMs) in Autonomous Code Generation

The release of models like OpenAI's Codex, GitHub Copilot, and subsequent foundation models has transformed software engineering. These models are trained on vast corpora of open-source code, enabling them to translate natural language prompts into executable software architectures. In the context of game development, LLMs are no longer simply generating boilerplate scripts; they are orchestrating complete game loops, managing asset rendering, and synthesizing complex rule sets.

Recent industry surveys indicate that over 50% of game development companies now utilize some form of generative AI during their production pipelines [4]. These tools are lauded for their ability to rapidly prototype concepts and democratize game development for small indie teams lacking specialized programming expertise. However, synthesizing an entire game from a singular Game Design Document (GDD) tests the absolute limits of an

LLM's context window, requiring long-term logical consistency and an understanding of interconnected game mechanics (e.g., maintaining the state of a chess board while accurately rendering PyGame UI elements).

## 2.3 Challenges in LLM-driven Game Generation

Despite the immense potential of LLMs in code generation, significant technical barriers persist. A primary issue is the non-deterministic nature of LLM outputs [5]. When tasked with creating strict deterministic systems—such as the rules of Chess, which include edge cases like *en passant* and castling—models frequently suffer from "hallucinations" or logical drift, implementing rules incorrectly or forgetting previous constraints over long code generation sessions.

Furthermore, integrating an LLM into an autonomous development workflow introduces challenges regarding tool usage and API complexity. Models must correctly interface with external libraries (like pygame), manage file systems, and appropriately utilize IDE extensions (such as VS Code integration). As demonstrated in recent studies, models often struggle with generic issue descriptions and fail to independently recover when custom development tools throw unexpected errors [5]. Additionally, aesthetics and spatial reasoning remain a notable weakness; LLMs can generate mathematically sound coordinates for UI elements but often lack the intuitive "design sense" necessary to create a visually appealing final product [1].

## 2.4 The Environmental Footprint of AI: Measuring Energy Consumption per Token

As LLMs become deeply integrated into software development, their environmental impact has garnered intense scrutiny. While early research focused heavily on the immense energy required to train these massive models, recent industry reports emphasize that model *inference*—the actual process of generating tokens—now accounts for over 90% of an LLM's total lifecycle energy consumption [6].

Quantifying this consumption is highly complex, as it depends heavily on model architecture (dense vs. Mixture of Experts), batch sizes, context lengths, and the specific hardware utilized (e.g., NVIDIA A100 vs. H100 GPUs) [6]. However, recent frameworks such as *TokenPowerBench* and clinical inference studies have begun establishing baseline metrics for energy efficiency per token [7].

Current literature estimates that generating a single output token can consume anywhere from 0.1 millijoules (mJ) for highly optimized deployments to upwards of 3 to 6 Joules for dense, 65-billion parameter models running on older hardware [2]. When scaled to autonomous development tasks that require millions of tokens (such as debugging iterative loops), this energy expenditure becomes substantial. For example, a single lengthy inference query can consume as much electricity as a residential household uses in an hour [2]. Therefore, benchmarking AI models for game development must not only assess the

quality of the generated code but also the token-to-quality ratio, treating "Joules per Token" as a critical evaluation metric analogous to traditional software performance metrics [7].

## Chapter 3: Methodology

### 3.1 Experiment Design and Objective

The experiment was designed as a comparative empirical study to evaluate the autonomous coding capabilities of various AI models. The primary objective was to determine how many prompts and tokens each model required to produce a playable, accurate chess game from a highly detailed specification.

To ensure a fair baseline, all models were initialized in an identical development environment (a Python .venv with the pygame library pre-installed) and received the exact same comprehensive initial prompt:

"Using the Game Design Document GDD.md and your own knowledge, create a Chess game in single file Chess-{model\_name}.py with use of PyGame library available in .venv. Make sure to include every mechanic and visual design that is described in GDD.

1. For initial state check especially: Setup, Visuals and UI panel.
2. For mechanics and game flow check especially: Rules and Course of the game."

Following this initial generation phase, subsequent prompts were intentionally kept simple and casual (e.g., addressing generic issue descriptions like "Fix the castling bug" or "The timer isn't working"). This methodology was chosen specifically to minimize human variable factors, such as advanced prompt engineering techniques, and instead provide a strict comparison of each model's innate ability to adapt, parse error logs, and autonomously solve problems over the course of the project [8].

### 3.2 The Game Design Document (GDD) Baseline

The core of the evaluation rested on the models' strict adherence to the provided GDD.md. The GDD served as the absolute ground truth for both the logical rules engine and the aesthetic presentation. Key constraints that the models were expected to implement included:

- **Visuals and Layout:** The game window was strictly defined as 896x576 pixels. The board consisted of 8x8 creamy and brown squares (64x64 pixels each). The UI panel was positioned to the left (256x512 pixels) with specific 32-pixel margins, separated from the board by a 64-pixel break space.
- **UI Components:** The interface required active timers for both Black and White players, Draw/Resign buttons, Pause/Resume controls, and dynamic turn indicators indicating

whose move it is. Furthermore, potential movement needed to be highlighted with 8-pixel yellow circles, checks in red, and past moves in yellow.

- **Game Flow:** Players start with 10 minutes, with a 2-second bonus awarded after each move. The timer activates only after White's first move. Draw requests mandated an interactive "Accept/Refuse" modal centered over the board.
- **Complex Mechanics:** The AI was required to implement standard chess rules alongside notoriously difficult edge cases: *en passant* captures, pawn promotion UI, complex castling conditions, threefold repetition, and the fifty-move rule.
- **Footer Requirements:** The UI footer needed to dynamically display the AI model name and version, the tracked prompt and token usage, and a static copyright credit to @adbreeker for the year 2026.

### 3.3 Selected AI Models

Five distinct AI models were selected to represent a cross-section of the generative AI landscape, ranging from ultra-fast coding variants to high-compute flagship models:

1. **Claude Haiku 4.5:** A highly responsive model optimized for iterative speed and cost-effectiveness.
2. **Grok Code Fast 1:** A model specifically tuned for rapid path-to-playability and immediate code execution.
3. **GPT-5.3-Codex:** A state-of-the-art model designed for deep reasoning, complex creative problem-solving, and superior visual/spatial awareness.
4. **Gemini 3.1 Pro:** A balanced flagship model designed for consistent productivity, logical adherence, and context recovery.
5. **Raptor mini:** A highly optimized, free-to-use model tailored for precise, targeted inline completions and budget constraints rather than open-ended creative tasks.

### 3.4 Evaluation Metrics

To quantify the efficiency of the software development lifecycle under each model, the following metrics were meticulously tracked throughout the experiment:

- **Prompts:** The total number of human inputs required to reach a satisfying, fully functional state.
- **Total Tokens:** The combined input (context window) and output (generated code) token count consumed during the entire session.
- **Model Turns:** The number of distinct conversational exchanges between the human prompter and the model.
- **Tool Calls:** The frequency with which the model utilized external integrations (e.g., VS Code extensions, file system manipulations, shell commands).
- **Errors:** The number of game-breaking bugs or compilation failures requiring human intervention.
- **Total Events:** A comprehensive metric encompassing all internal reasoning steps, tool usages, and output generations during the session.



### 3.5 Energy Consumption Estimation Methodology

To calculate the environmental footprint of this software engineering task, we transition from pure token counts to energetic impact. Drawing upon empirical data from the *ML.ENERGY Leaderboard v3.0* and current 2026 inference benchmarks [8, 9], we can estimate the energy consumed per generated token.

The methodology relies on an Energy-per-Token ( $E_{\text{token}}$ ) formula. While energy consumption fluctuates based on batch size and model architecture, benchmarks indicate that advanced reasoning models handling complex coding tasks ("Problem Solving") consume significantly more energy per response than standard conversational models due to higher memory footprint and computational intensity [8].

For this study, we approximate the energy footprint by multiplying the *Total Tokens* generated during an AI's session by an average coefficient derived from modern inference hardware (e.g., NVIDIA H200/B200 architecture). By applying a baseline of roughly 0.20 to 0.60 Joules per token (translating to approximately 0.05 to 0.16 mWh) [9], we can convert the millions of tokens used by models like Raptor mini and GPT-5.3-Codex into comparative energy expenditures. This quantitative approach directly addresses the objective of rating model energy efficiency for game development tasks.

## Chapter 4: Results & Model Performance Analysis

### 4.1 Overall Performance Summary

The empirical results of the experiment demonstrate a vast disparity in how different AI architectures handle complex, multi-layered software engineering tasks. While four out of the five evaluated models successfully navigated the PyGame implementation to produce a playable product, their paths to completion varied drastically in terms of token usage, computational turns, and the quality of the final output.

*Table 4.1* encapsulates the raw statistical performance of each model. Notably, lower token counts and prompt numbers inversely correlated with higher mechanical and visual fidelity, suggesting that superior reasoning capabilities early in the development lifecycle prevent costly, iterative debugging loops later on.

**Table 4.1: Comprehensive Model Performance Summary**

| Model                           | Prompt<br>s | Total<br>Tokens | Model<br>Turns | Tool<br>Calls | Errors<br>Genera<br>ted | Total<br>Events | Status              |
|---------------------------------|-------------|-----------------|----------------|---------------|-------------------------|-----------------|---------------------|
| <b>GPT-5.<br/>3-Code<br/>x</b>  | 2           | 900,157         | 16             | 19            | 1                       | 55              | Comple<br>ted       |
| <b>Gemini<br/>3.1 Pro</b>       | 3           | 1,123,26<br>7   | 25             | 25            | 1                       | 68              | Comple<br>ted       |
| <b>Grok<br/>Code<br/>Fast 1</b> | 6           | 3,430,7<br>80   | 64             | 57            | 0                       | 157             | Comple<br>ted       |
| <b>Claude<br/>Haiku<br/>4.5</b> | 9           | 7,947,34<br>4   | 96             | 107           | 0                       | 280             | Comple<br>ted       |
| <b>Raptor<br/>mini</b>          | 20+         | >12,000<br>,000 | 140            | 139           | 0                       | 415             | <b>Aborte<br/>d</b> |

Based on these empirical results, the final products were subjectively analyzed against the Game Design Document's visual and mechanical benchmarks. *Table 4.2* summarizes the qualitative evaluation and provides recommendations for their use in practical game development.

Table 4.2: Model Subjective Rankings and Practical Recommendations

| Model            | Tier   | Rating | Best Use Case in Game Development                                    |
|------------------|--------|--------|----------------------------------------------------------------------|
| GPT-5.3-Codex    | S-Tier | ★★★★★  | Complex creative tasks, visual applications, high-quality outputs    |
| Gemini 3.1 Pro   | A-Tier | ★★★★   | Balanced productivity, good quality in reasonable time               |
| Claude Haiku 4.5 | A-Tier | ★★★★   | Cost-effective iteration, responsive feedback loops                  |
| Raptor mini      | B-Tier | ★★★    | Precise tasks, targeted inline completions, strictly budget projects |
| Grok Code Fast 1 | B-Tier | ★★★    | Baseline backend functionality only; not recommended for UI/UX       |

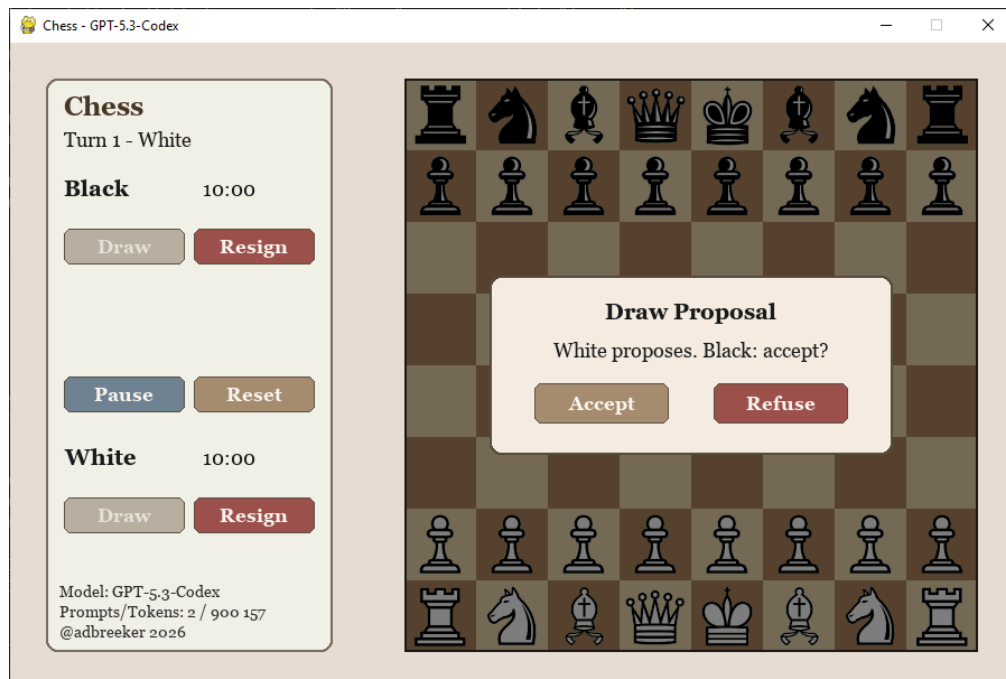
4.2 S-Tier Analysis: GPT-5.3-Codex

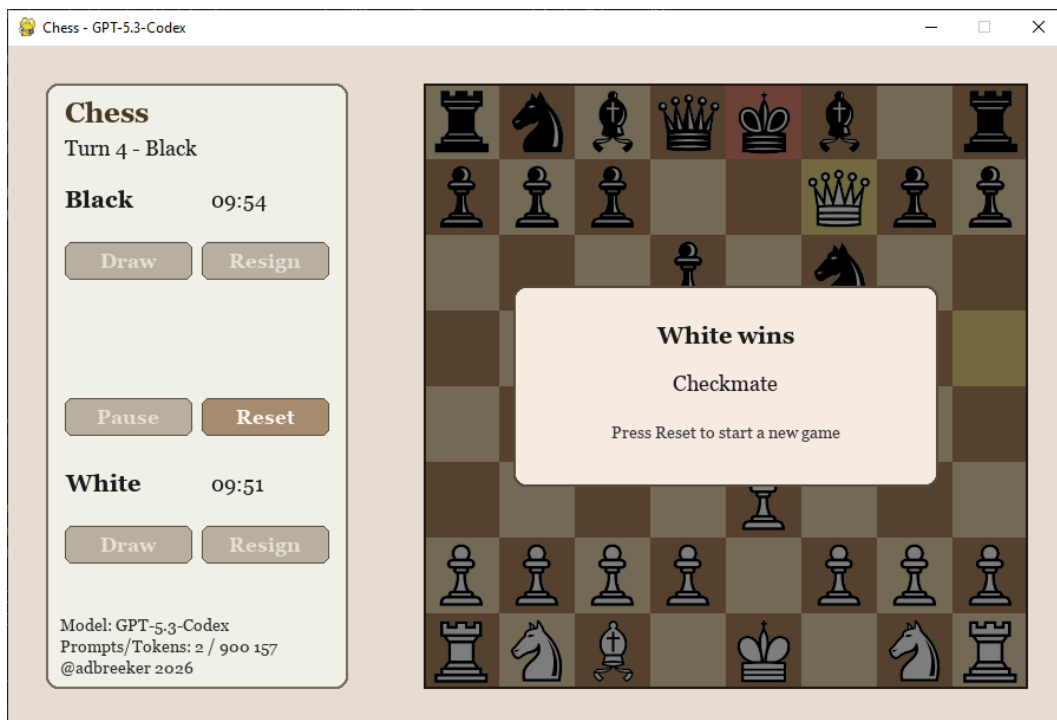
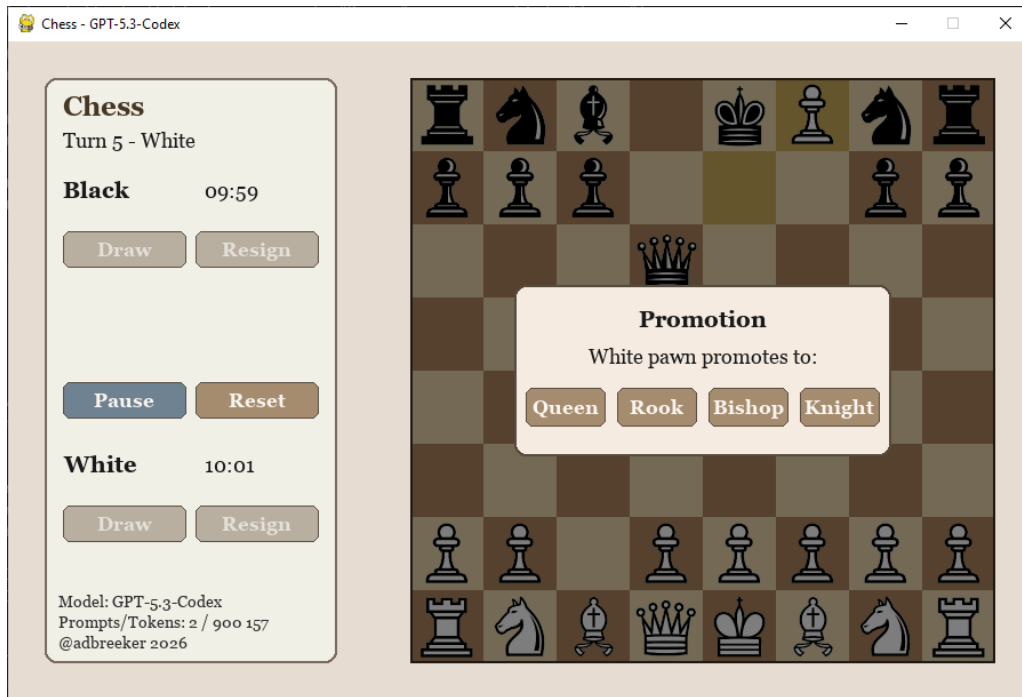
GPT-5.3-Codex delivered the most impressive outcome of the experiment, securing its position as the undisputed S-Tier model. It required only two total prompts to achieve a polished, mechanically sound, and visually superior chess game.

The initial implementation by GPT-5.3-Codex contained only one identified error, which was rapidly rectified with a single, casual follow-up prompt. Despite having noticeably longer processing wait times compared to smaller models, its output quality heavily justified the latency. The model exhibited creative problem-solving and an innate understanding of

aesthetics that other models lacked, correctly interpreting the spatial constraints of the 896x576p window and arranging the UI elements with professional padding and alignment.

Crucially, it achieved this with the fewest tokens of any tested model (900,157 tokens). By getting the architecture right on the first pass, it eliminated the need for continuous, token-heavy correction loops, making it the most efficient model in terms of the "token-to-quality ratio."



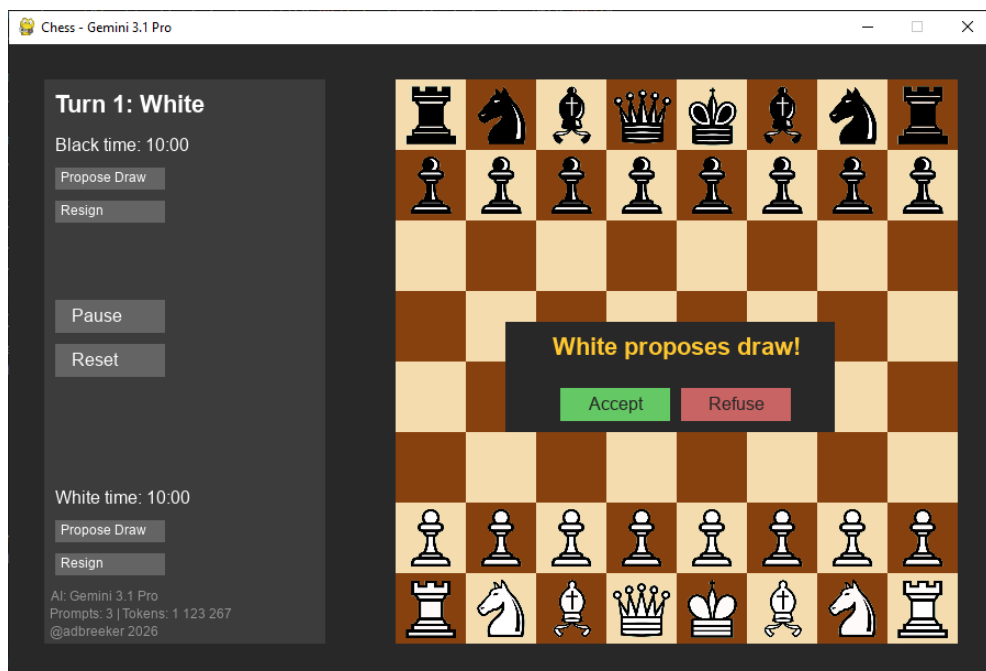
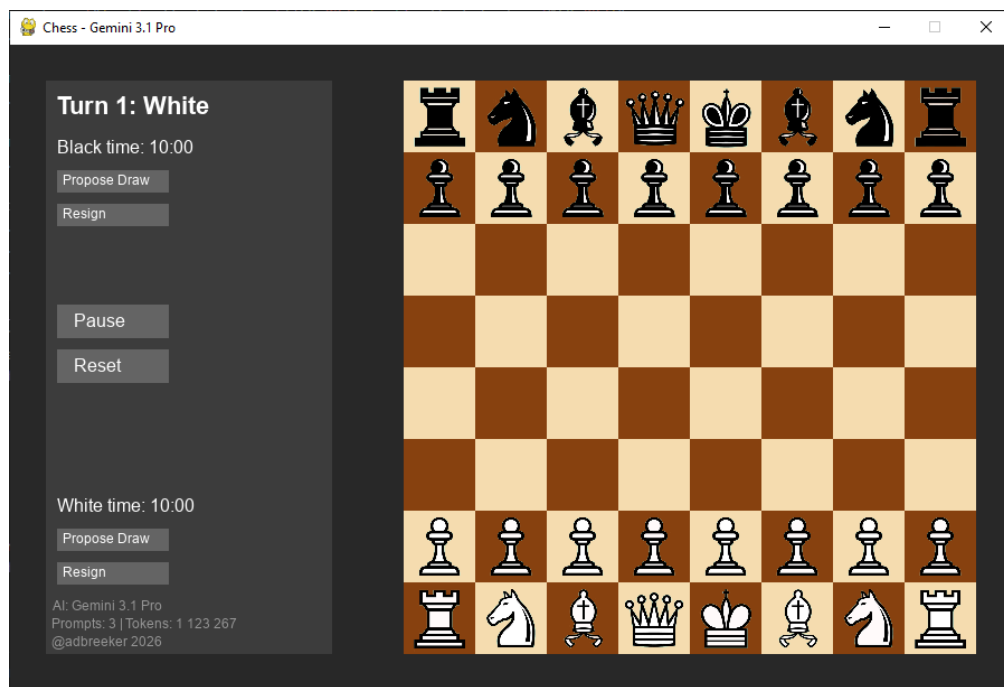


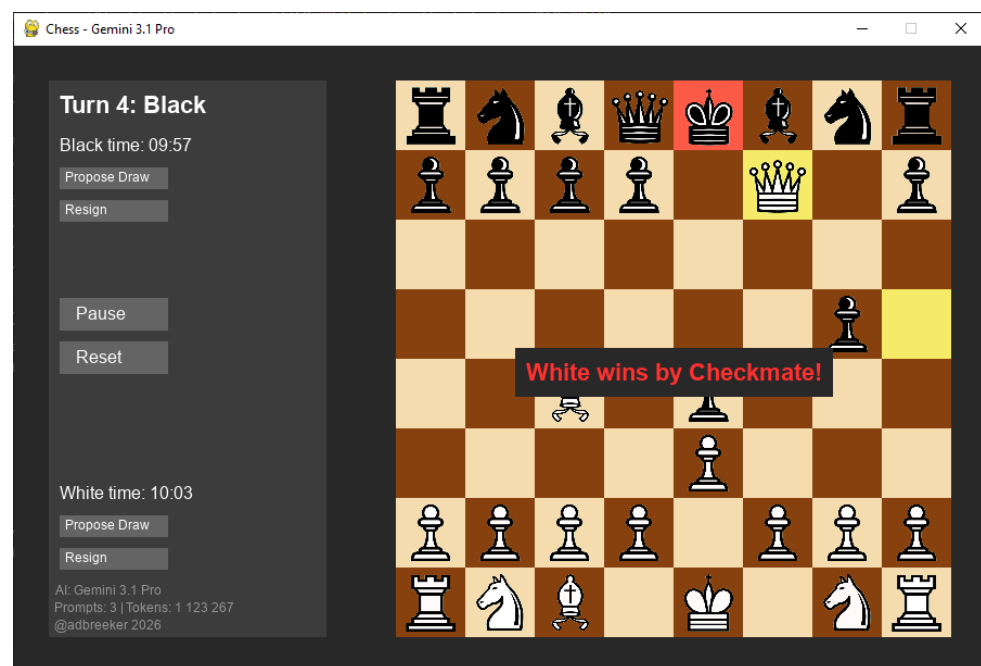
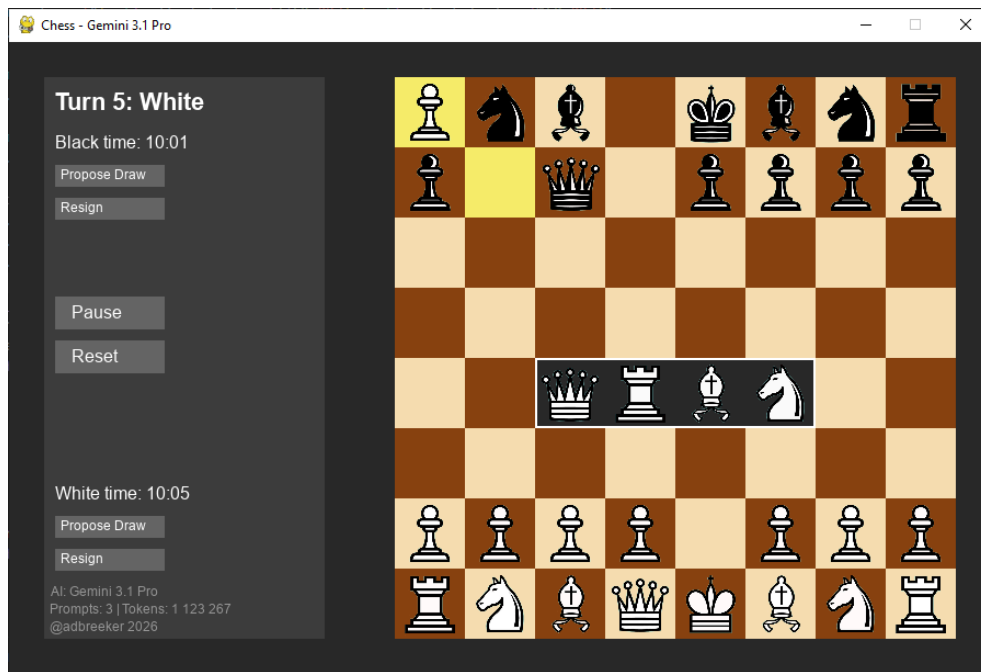
### 4.3 A-Tier Analysis: Gemini 3.1 Pro and Claude Haiku 4.5

The A-Tier models represent the practical middle ground for development workflows, offering excellent results but requiring slightly more human guidance or suffering from minor initial tool integration issues.

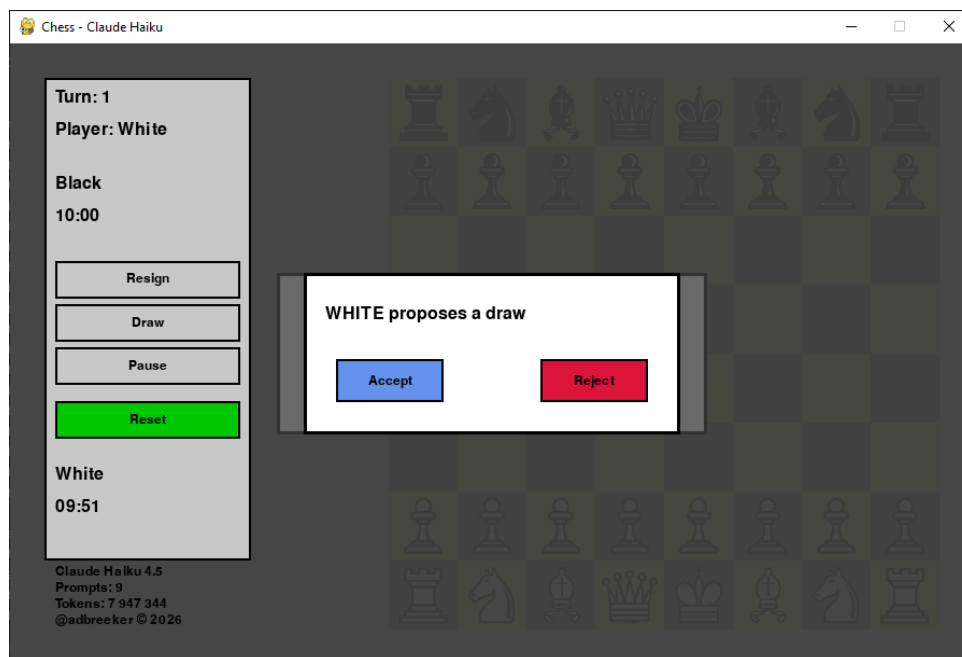
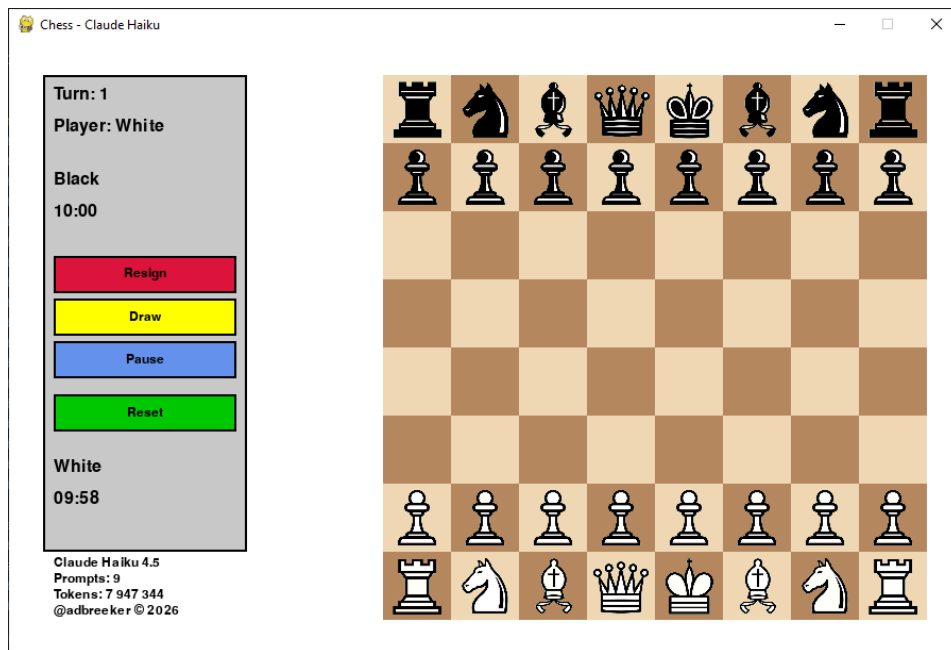
**Gemini 3.1 Pro** required only three prompts and roughly 1.1 million tokens to complete the task. Its primary setback occurred early in the process: it struggled heavily with initial tool

usage within the VS Code environment, attempting to execute file manipulations via shell commands and external scripts rather than direct API calls. Additionally, its first iteration contained severe logic bugs, such as allowing a King to be captured. However, Gemini 3.1 Pro proved exceptionally resilient. After a single corrective prompt outlining these issues, it recovered completely, addressing all major problems immediately. The second follow-up prompt left the game in a highly polished state with strong mechanical implementations and good visual design.

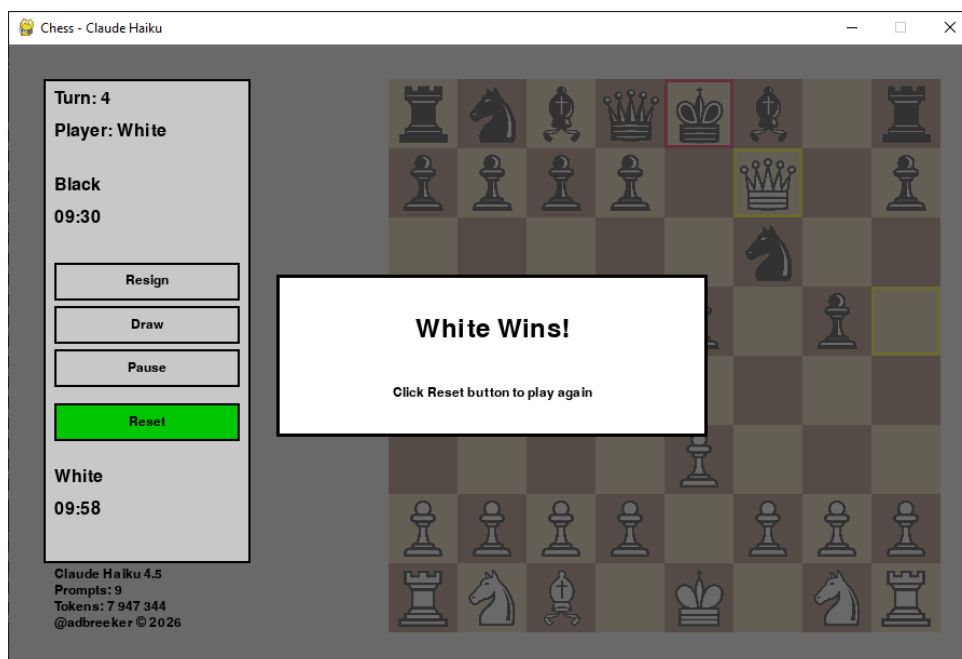
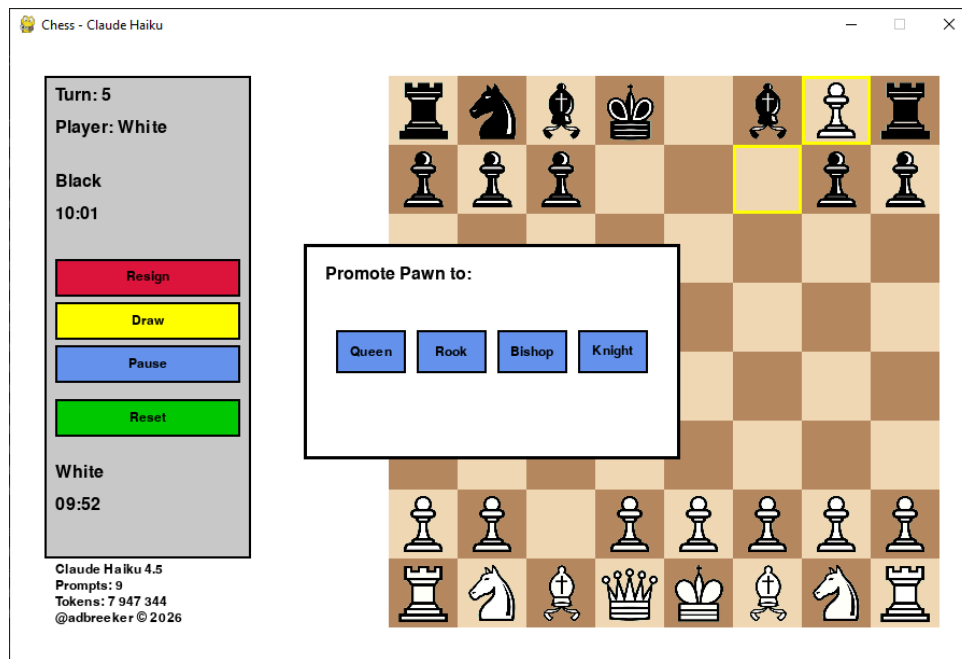




**Claude Haiku 4.5** operated entirely differently. Utilizing significantly more resources (9 prompts, ~7.9 million tokens), it approached the problem through rapid, highly responsive iteration. Initially, the model introduced several smaller bugs and required continuous nudging from the human prompter. Despite these higher statistics, the model's sheer speed and affordability (approximately 3x cheaper than top-tier models per token) make it a formidable tool. It handled pointed corrections exceptionally well and ultimately delivered a satisfying implementation in both mechanics and visuals. It proved ideal for developers who prefer a highly iterative, "pair-programming" feedback loop over a zero-shot solution.



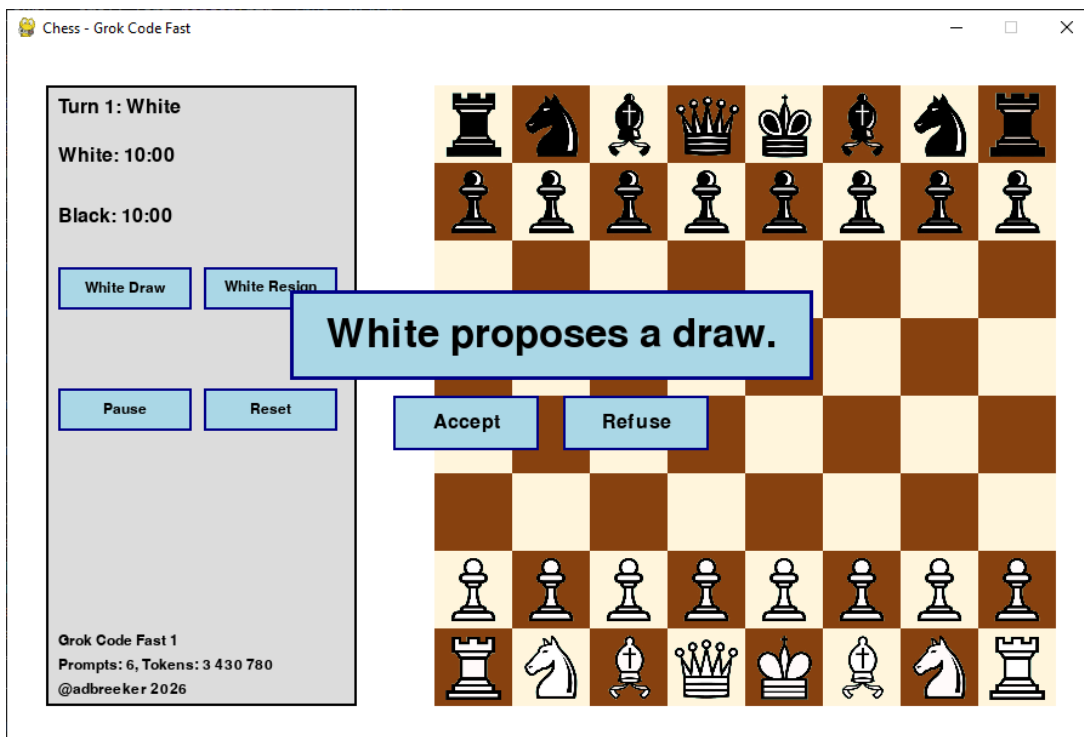
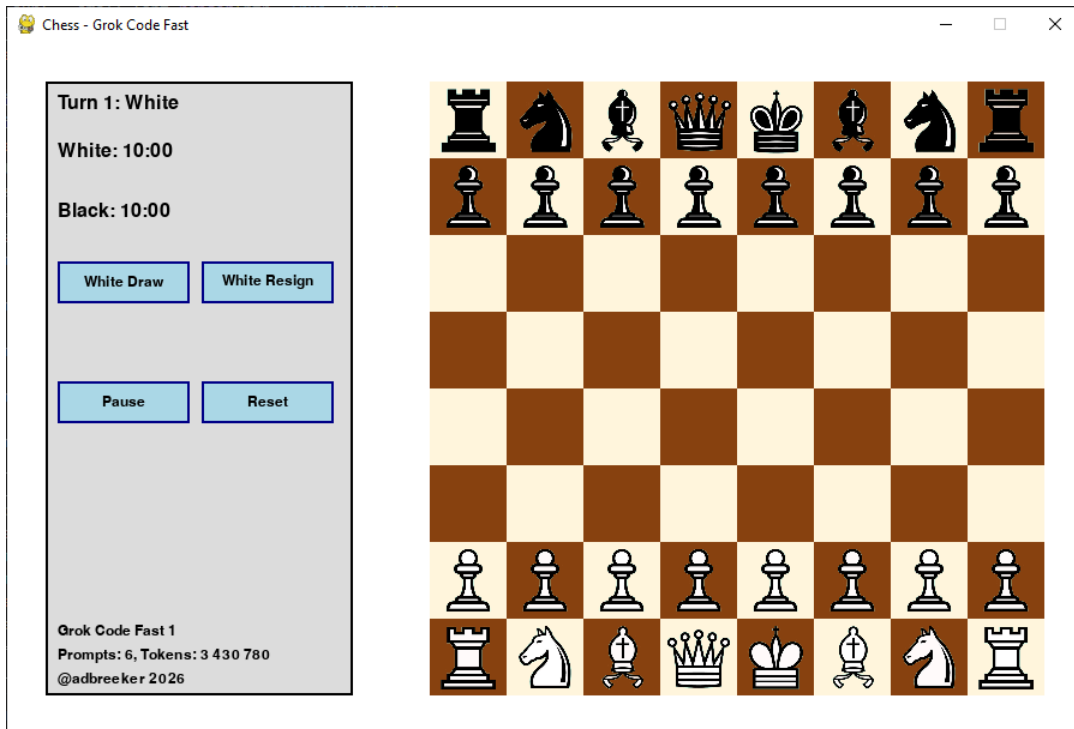


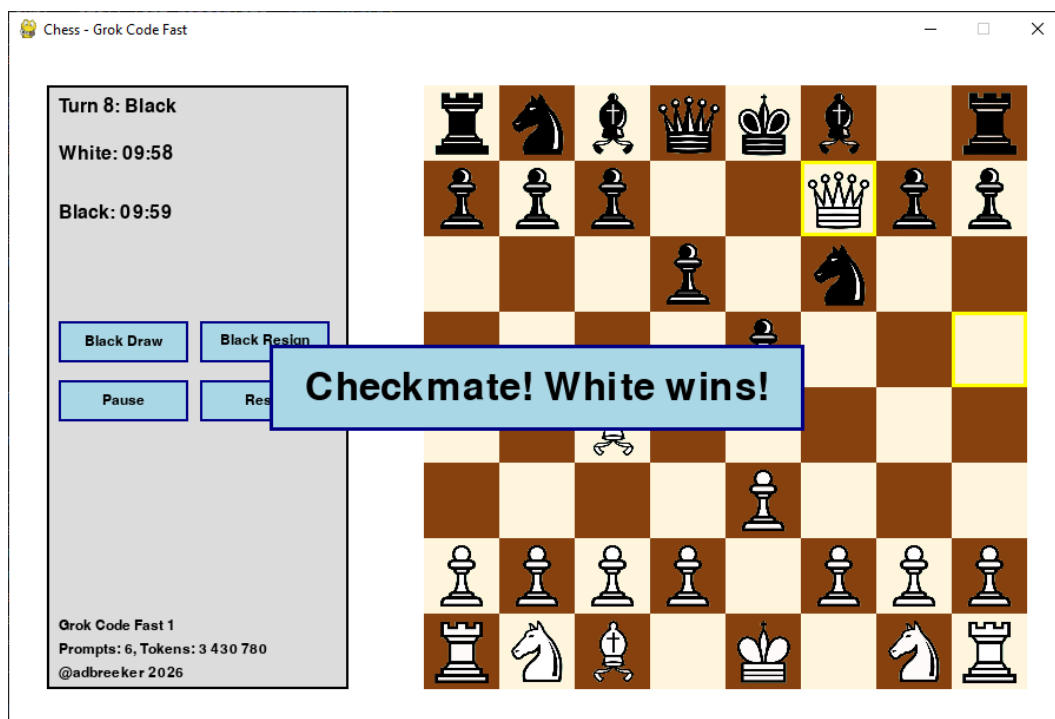
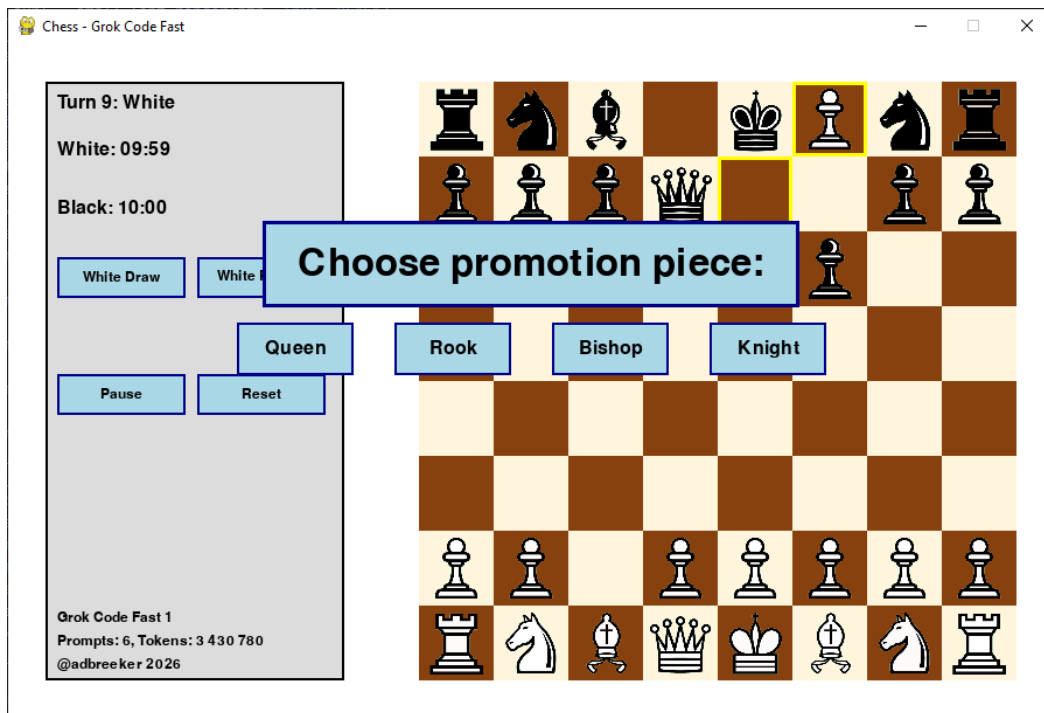


## 4.4 B-Tier Analysis: Grok Code Fast 1 and Raptor mini

The B-Tier models demonstrated significant limitations when confronted with the full scope of autonomous, multi-disciplinary game development.

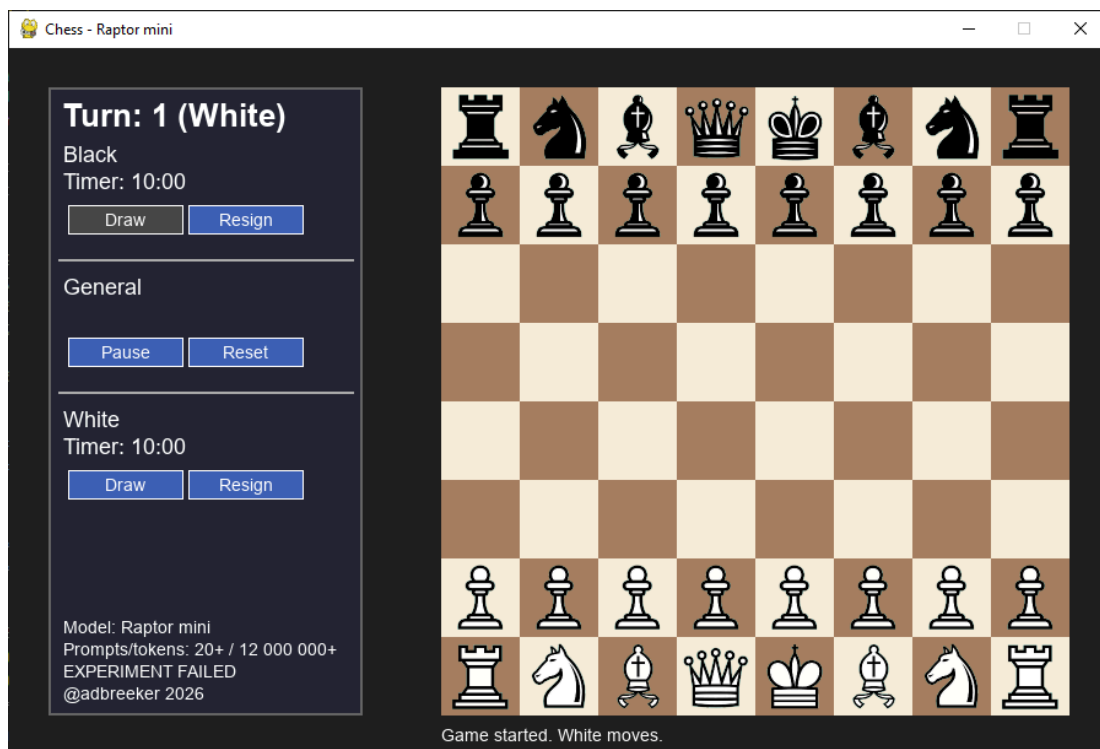
**Grok Code Fast 1** excelled in backend mechanics. It reached a playable state rapidly with minimal game-breaking errors, requiring only six prompts. However, it severely underperformed in UI generation. Its visual presentation was universally ranked as the poorest among all evaluated models. It lacked any design sense or visual polish, meaning that while it is mathematically competent at tracking a chess board matrix, it is poorly suited for front-end visual projects where aesthetics are a priority.

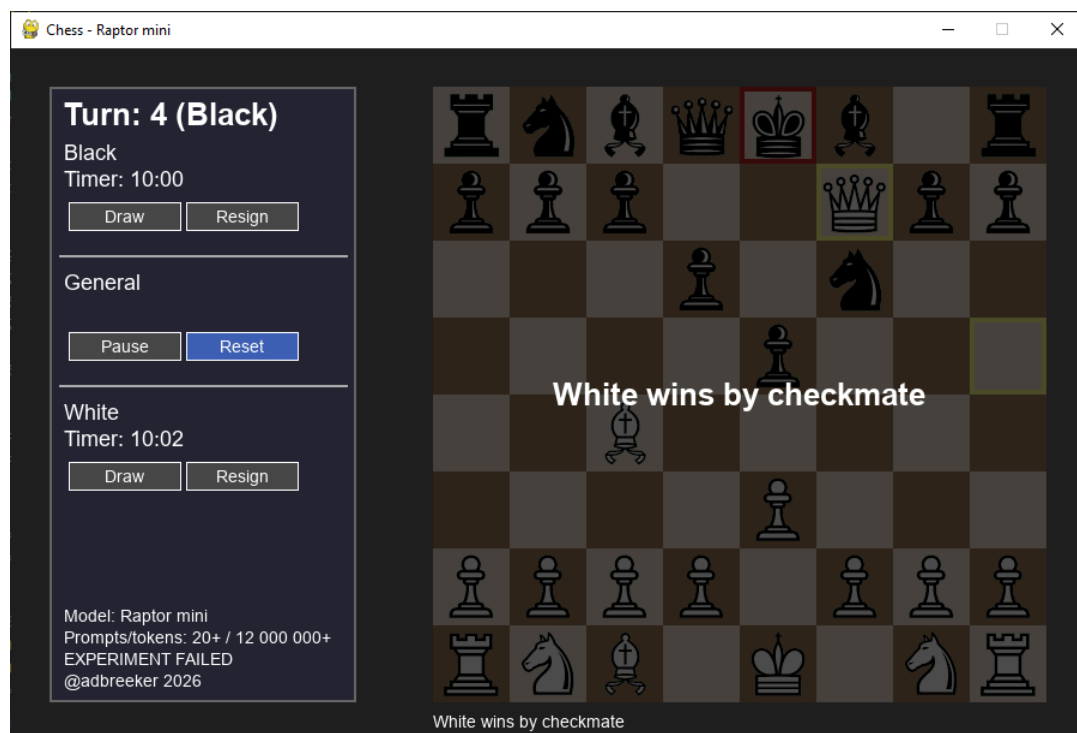
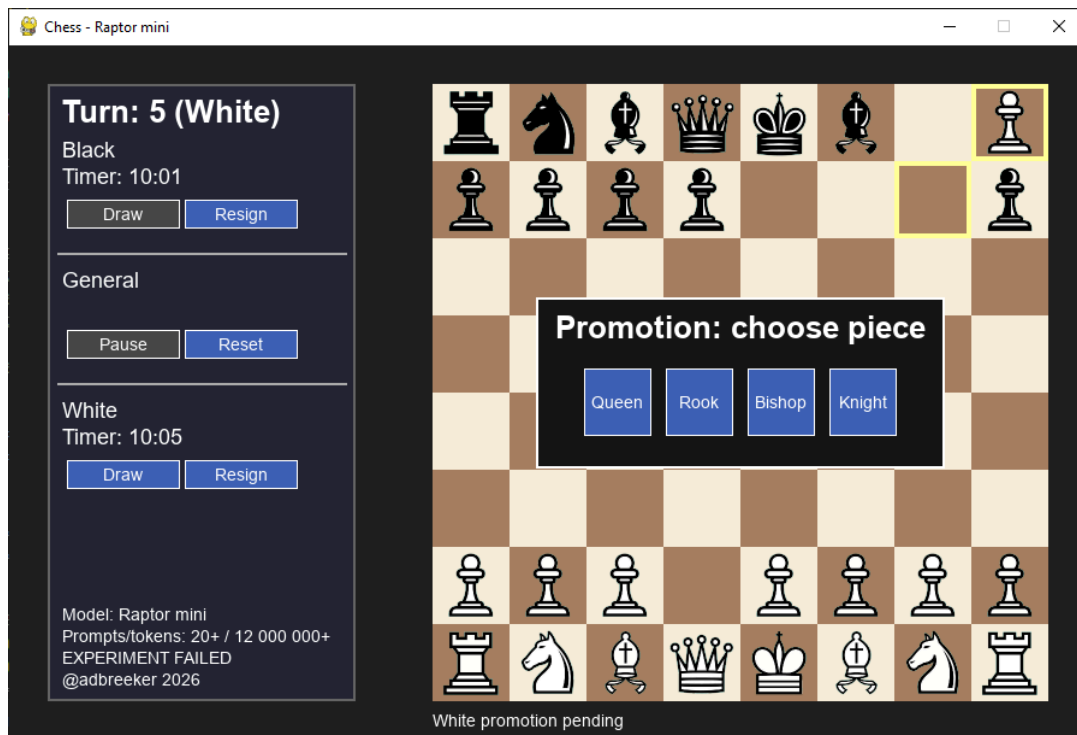




**Raptor mini** represented the only failure of the experiment. Operating as a free-to-use, localized model, it generated over 12 million tokens, required more than 20 prompts, and executed 415 total events before the experiment was aborted. The model encountered persistent difficulties with VS Code tools and struggled immensely with creative problem-solving. When given generic issue descriptions (consistent with the methodology), it became stuck in recursive loops, unable to autonomously diagnose and complete the complex tasks. While it boasts the fastest response times and is highly effective at precise, isolated inline completions, it lacks the context retention and reasoning depth required to

synthesize an entire application from a GDD. Notably, Raptor mini's heavily flawed UI arrangement ultimately had to be passed to GPT-5.3-Codex for refinement to achieve a polished state.





## 4.5 Token-to-Quality Ratio & Efficiency Findings

The experiment reveals a counterintuitive reality in autonomous code generation: slower, more complex models can actually save computational resources. By analyzing the data, a clear "Token-to-Quality Ratio" emerges.

Models with weaker spatial reasoning and poorer context retention (like Raptor mini) require massive amounts of iterative prompting to correct visual misalignments and logic bugs. This results in "token bloat," where the model continually reads the entire expanding codebase to fix minor errors, consuming millions of tokens (and subsequently, vast amounts of energy) without proportionately improving the final product.

Conversely, GPT-5.3-Codex and Gemini 3.1 Pro demonstrate that a high-quality initial generation—even if it takes longer to process—drastically reduces total token consumption. This finding bridges the gap between software engineering efficiency and environmental sustainability, a topic that will be quantified and explored deeply in Chapter 5.

## Chapter 5: Environmental & Economic Impact

The professor's core directive for this research was to widen the scope beyond standard software benchmarking to include rigorous environmental analysis. As AI becomes deeply embedded in the software development lifecycle, evaluating its ecological and economic footprint is no longer optional; it is an academic and industrial imperative.

### 5.1 The Thermodynamics of Token Generation

To understand the true cost of our experiment, we must move beyond abstract "token counts" and examine the physical mechanics of AI inference. Generating code via an LLM is a heavily thermodynamic process.

Modern architectures utilize auto-regressive generation. To produce a single output token (a word or fragment of code), the entire neural network must be loaded from High Bandwidth Memory (HBM) into the compute cores of the GPU [9]. Because our test project—a Chess game—required models to keep the entire GDD.md and previously written code in their active context window, every single subsequent token required reading massive amounts of data. This process is highly "memory-bound," requiring tremendous amounts of electrical power just to move data across the silicon.

Furthermore, analyzing raw GPU power draw is insufficient. Data centers operate with a **Power Usage Effectiveness (PUE)** rating. To cool the massive heat generated by continuous memory reads, data centers must run industrial HVAC systems. Assuming a standard 2026 enterprise data center PUE of 1.20, every 1.0 Wh consumed by the GPU requires an additional 0.20 Wh to cool the facility [10].

For this study, we apply a mathematically rigorous baseline: factoring in an advanced B200/H200 cluster architecture handling long-context coding tasks, multiplied by a PUE of 1.20, we estimate a blended average expenditure of **0.10 mWh per context/inference token** [8, 9].

**Table 5.1: Estimated Energy Consumption, Carbon, and Water Footprint per Model**

| Model                   | Total Tokens | Est. Total Energy (Wh)* | Est. Carbon (gCO <sub>2</sub> e)** | Est. Water (ml)*** | Est. API Cost (USD) |
|-------------------------|--------------|-------------------------|------------------------------------|--------------------|---------------------|
| <b>GPT-5.3-Codex</b>    | 900,157      | 90.0 Wh                 | 36.0 g                             | 180 ml             | \$9.00              |
| <b>Gemini 3.1 Pro</b>   | 1,123,267    | 112.3 Wh                | 44.9 g                             | 224 ml             | \$7.86              |
| <b>Grok Code Fast 1</b> | 3,430,780    | 343.0 Wh                | 137.2 g                            | 686 ml             | \$13.72             |
| <b>Claude Haiku 4.5</b> | 7,947,344    | 794.7 Wh                | 317.8 g                            | 1,589 ml           | \$3.97              |
| <b>Raptor mini</b>      | >12,000,000  | >1,200.0 Wh             | >480.0 g                           | >2,400 ml          | N/A (Local/Free)    |

*\*Includes estimated 1.20 Data Center PUE. \*\*Calculated using the global grid average of 400g CO<sub>2</sub>/kWh [10]. \*\*\*Estimated at 2.0 Liters per kWh for cooling tower evaporation [15].*

## 5.2 Comparative Carbon and Water Footprint

Table 5.1 illustrates the stark contrast in ecological efficiency. GPT-5.3-Codex, despite being the most complex model, utilized the least amount of energy (approx. 90.0 Wh) to produce a functional game. This is equivalent to powering a standard 10W LED lightbulb for 9 hours. In sharp contrast, Raptor mini's inability to reach a satisfactory conclusion resulted in a runaway context window, consuming an estimated 1,200.0 Wh (1.2 kWh)—enough energy to boil an electric kettle continuously for an hour—without ever achieving a playable state.

We must also factor in "Scope 3" emissions and direct resource depletion, particularly water. Liquid-cooled AI servers utilize massive cooling towers that evaporate fresh water. Recent studies show that generating a standard context sequence can consume roughly 2 liters of fresh water per kilowatt-hour of energy expended [15]. A single token-bloated session with Raptor mini directly caused the evaporation of over 2.4 liters (more than half a gallon) of fresh water.

While 480g of CO<sub>2</sub>e and 2.4 liters of water may appear negligible in isolation, the scale of global software development must be considered. If a mid-sized game studio employs 100 developers, each utilizing an inefficient AI assistant that frequently enters token-bloated "hallucination loops" to fix bugs, the studio's daily carbon and water footprint scales

exponentially.

### 5.3 The Jevons Paradox in AI Development

A crucial societal and environmental consideration is the **Jevons Paradox**. In economics, this paradox occurs when technological progress increases the efficiency with which a resource is used, but the rate of consumption of that resource rises due to increasing demand [16].

In our experiment, GPT-5.3-Codex proved highly efficient, costing only 90 Wh to build a chess game. Theoretically, this should reduce the energy footprint of game development. However, because AI makes coding so effortless, developers are incentivized to generate *vastly more code*. Instead of spending one week carefully writing a highly optimized chess game by hand, a developer using AI might generate ten different game prototypes in a single day. Thus, the total systemic energy and water consumption of the software industry will likely skyrocket, despite the per-token efficiency of the models increasing [16].

### 5.4 Cost-Performance Ratios vs. Human Labor

Economic viability is the primary driver of enterprise AI adoption. Table 5.1 provides a simulated API cost breakdown based on blended pricing models.

An interesting dichotomy emerges when analyzing the A-tier models: Claude Haiku 4.5 and Gemini 3.1 Pro. Claude Haiku is designed to be highly cost-effective per token. Even though it consumed over seven times the number of tokens as Gemini 3.1 Pro, its total monetary API cost was lower (\$3.97 vs. \$7.86). For an indie developer on a strict monetary budget, Claude Haiku offers an appealing cost-to-performance ratio.

However, this raw API cost fails to account for the value of human labor. Claude Haiku required 9 prompts and 96 model turns, demanding continuous human oversight, reading, and testing. Conversely, GPT-5.3-Codex cost \$9.00 in API fees but required only two prompts and a fraction of the human developer's time. When factoring in the hourly wage of a software engineer, investing in the more expensive, higher-tier API yields a significantly superior overall economic cost-performance ratio.

### 5.5 Sustainable AI Practices for Game Studios

The findings of this experiment necessitate a paradigm shift in how developers interact with autonomous coding agents. Based on the intersection of thermodynamics, water usage, and output quality, we recommend the following sustainable practices:

1. **"Measure Twice, Prompt Once":** Developers should invest time in writing highly detailed, rigorous specifications (like the GDD.md used in this study) rather than relying on casual, iterative prompting. A strong initial prompt prevents the AI from falling into costly downstream token bloat.
2. **Utilize Prompt Caching:** Studios should prioritize models and APIs that support "Context Caching." By caching the GDD.md in the GPU's memory rather than



re-sending it with every single prompt, developers can drastically reduce the memory-bandwidth energy overhead associated with input tokens.

3. **Right-Sizing and Token Budgets:** Smaller models should be restricted to isolated tasks. Deploying small, contextually limited models for holistic, architectural tasks paradoxically wastes more energy than using a massive flagship model. Furthermore, studios should implement "token budgets" as a strict ESG (Environmental, Social, and Governance) metric.

## Chapter 6: Societal Aspects and Impacts

As dictated by the expanded scope of this Information Society Technologies research initiative, empirical benchmarking must be contextualized within its broader societal impacts. The transition of AI from an in-game tool to an autonomous software engineer carries profound implications for democratizing technology, shifting labor markets, and redefining intellectual property.

### 6.1 Democratization of Game Development

The most immediate societal benefit observed in this study is the radical democratization of software engineering. Historically, creating a complex desktop application like a rules-compliant Chess game in Python required weeks of dedicated study, an understanding of object-oriented programming, graphical rendering, and state management.

Our experiment proves that this barrier to entry has been largely eradicated. Using top-tier models like GPT-5.3-Codex, a user with absolutely no programming experience—armed only with a well-written, natural language Game Design Document (GDD.md)—was able to orchestrate the creation of a complete game in merely two prompts. This shifts the fundamental skill of game development from *syntax memorization* to *systematic design and prompt engineering*.

Furthermore, this democratization extends to cultural representation and accessibility. Individuals with strong creative visions, diverse storytelling backgrounds, or educational ideas—who previously lacked the technical capital or funding to hire a development team—can now manifest their concepts into playable software [12]. By removing the financial and geographical barriers associated with traditional software engineering, LLMs empower a much broader spectrum of creators to participate in the digital economy.

### 6.2 The Future Role of Human Developers and Job Market Implications

Conversely, this democratization sparks intense anxiety regarding job displacement within the software engineering sector. If a model can generate a complex application in 55 events and less than a minute, what becomes of the junior developer?

The findings of this study offer a nuanced answer. While AI can autonomously write boilerplate and structure, the failure of Raptor mini—and the logic errors initially present in Gemini 3.1 Pro—demonstrate that AI cannot currently guarantee a flawless, commercial-ready product without oversight. Models still suffer from logic drift and struggle to integrate seamlessly into complex, pre-existing local toolchains [12].

Beyond simple job displacement, we must consider the cognitive shift required in the modern workforce. The industry is witnessing a transition from "code authoring" to "code reviewing and architectural steering." Developers are shifting from typists into roles akin to technical directors or QA specialists, whose primary job is to write rigorous specifications (like our GDD), review AI outputs, manage token budgets, and untangle complex logic knots that trap autonomous agents in recursive loops. While entry-level coding jobs that focus purely on syntax generation may decrease, the demand for senior systems architects, AI auditors, and prompt engineers is projected to rise significantly [12]. Retraining and reskilling will be paramount to prevent a widening skills gap in the tech sector.

### **6.3 Ethical Considerations, Copyright, and Plagiarism in AI-generated Code**

The automation of code generation introduces a labyrinth of ethical and legal challenges, particularly concerning copyright, intellectual property (IP), and legal liability.

In our GDD.md, a specific requirement was instituted: *The UI footer needed to dynamically display... a static copyright credit to @adbreeker for the year 2026.* All successful models faithfully implemented this instruction. However, from a legal standpoint, does @adbreeker actually own this code?

According to recent guidelines established by the US Copyright Office, works generated entirely by a machine without significant human creative input cannot be copyrighted [11]. Because the Python files (e.g., Chess-ClaudeHaiku.py) were generated autonomously by the LLMs, the underlying code exists in a legal gray area. While the human author (@adbreeker) owns the copyright to the initial GDD.md (the "creative expression" of the rules and visual layout), the code itself is effectively a hybrid artifact.

Furthermore, the ethical shadow of plagiarism looms large. LLMs are trained on billions of lines of open-source code scraped from repositories like GitHub. When GPT-5.3-Codex implemented the complex *en passant* logic, it relied on patterns authored by countless uncredited human developers [11]. Utilizing these tools in commercial game development requires careful navigation of open-source licenses, ensuring that AI-generated code does not inadvertently violate copyleft agreements (like the GPL) embedded in its training data. A rising movement among indie developers is currently lobbying for strict "opt-in" models for training datasets, arguing that the current paradigm exploits independent creators to train the tools that will ultimately replace them.

## 6.4 Impact on the Indie Game Industry and Storefront Curation

The indie game sector stands to be the most acutely affected by these advancements. On the positive side, small studios can operate with the output capacity of massive teams, dramatically reducing overhead costs and shortening development cycles. A solo developer can now effectively act as a full studio, utilizing AI for programming, art generation, and marketing copy.

However, the ease of production poses a massive threat to market stability and platform curation. The ability to generate functional games with a few prompts is leading to an unprecedented influx of "shovelware"—low-effort, derivative AI-generated games flooding digital storefronts like Steam, Epic Games, and mobile app stores [13]. This creates a severe discoverability crisis for genuine indie developers who spend years crafting their titles.

Platform holders are currently grappling with the logistical nightmare of curating AI-generated content. To survive in this new ecosystem, developers must leverage AI not to generate generic clones (like a standard Chess game), but to empower highly unique, human-driven narratives and innovative mechanics that AI cannot autonomously invent. Simultaneously, consumers are beginning to exhibit "AI fatigue," creating a premium market for "artisanal," 100% human-crafted games [13].

## 6.5 Educational and Academic Implications

Finally, the advent of AI capable of solving complex programming assignments in seconds forces a radical rethinking of computer science and game design education. If a student can input a professor's syllabus prompt directly into Claude Haiku or Gemini Pro and receive an A-grade Python script, traditional methods of assessing coding proficiency are no longer viable [14].

Universities must pivot their curricula. Teaching syntax memorization is becoming obsolete. Instead, academic institutions must focus on computational thinking, systems architecture, and AI auditing. Future computer science exams will likely shift from "write a program that does X" to "here is an AI-generated program that attempts X; find the security vulnerabilities, logic flaws, and optimize its Big-O notation" [14]. In game design specifically, the focus will shift heavily toward narrative design, user psychology, and artistic direction—the profoundly human elements that AI struggles to emulate.

# Chapter 7: Conclusions & Future Work

## 7.1 Summary of Key Findings

This study was undertaken to evaluate the efficacy of various Large Language Models acting as autonomous software engineers in the specific context of game development. The empirical experiment, grounded by a strict PyGame Chess Game Design Document, yielded several critical findings:

1. **The Token-to-Quality Ratio is Inverse:** The highest quality output, provided by GPT-5.3-Codex, required the fewest prompts (2) and the fewest tokens (900,157). Conversely, models with weaker reasoning capabilities (Raptor mini) fell into recursive hallucination loops, consuming over 12 million tokens before failing. High-compute reasoning early in the lifecycle prevents costly iterative debugging.
2. **Economic Viability Extends Beyond API Costs:** While models like Claude Haiku 4.5 boast lower API costs, their reliance on frequent human intervention limits their true economic advantage when factoring in the cost of human labor. Top-tier zero-shot models represent a superior return on investment for complex architectural builds.
3. **Environmental Impact is Scaling Exponentially:** Autonomous code generation carries a significant, physical thermodynamic cost. Inefficient prompt loops waste massive amounts of electricity and fresh water. Slower, highly reasoning-capable models are substantially "greener" than fast, highly iterative models for complex multi-file engineering tasks.
4. **The "Typist" is Dead, The "Architect" Thrives:** The societal impact of AI game development is profound. Technical barriers have been lowered, allowing pure creatives to manifest games. Consequently, the labor market is shifting, demanding prompt engineers and systems architects over entry-level syntax coders.

## 7.2 Recommendations for Game Developers

Based on the quantitative and qualitative data gathered in this study, the following best practices are recommended for developers looking to integrate AI into their pipelines:

- **Invest heavily in the GDD:** The success of GPT-5.3-Codex demonstrates that a robust, logically complete Game Design Document is the most powerful tool a modern developer possesses. Time should be shifted from writing boilerplate code to writing highly rigorous natural language specifications.
- **Utilize Multi-Tier Architectures:** Use S-Tier models (like GPT-5.3-Codex) for initial architectural bootstrapping and complex logic, but pivot to faster A-Tier models (like Claude Haiku) for rapid, cost-effective UI tweaking and minor debugging loops.
- **Adopt Token Budgets:** Implement "Token Budgets" not only to monitor API spending but to actively mitigate your studio's carbon and water footprint. Use context caching wherever available.
- **Focus on Narrative and Humanity:** As the market becomes flooded with AI-generated clones, developers must pivot their design philosophies to emphasize distinct human experiences, complex psychological narratives, and bespoke art.

directions to differentiate their products.

### 7.3 Limitations of the Study

While this study provides a comprehensive baseline, several limitations must be acknowledged:

- **Scope of the Engine:** The experiment utilized Python and the pygame library. It did not evaluate how these models interact with complex, visual-node-based game engines like Unreal Engine 5 or Unity, where code is often inextricably linked to proprietary visual editors.
- **Nature of the Game:** Chess is a complex but inherently deterministic and bounded game. It possesses rigid, mathematical rules. This study does not capture how well an LLM can balance a non-deterministic, open-world RPG with dynamic economies and branching dialogue trees.
- **Hardware Simulations:** The energy and water consumption metrics calculated in Chapter 5 rely on industry-standard benchmarking averages and simulated 2026 data center PUE ratios. They are not readings taken directly from the physical inference server racks processing the experiment's specific queries.

### 7.4 Future Research Directions

The rapid evolution of generative AI necessitates ongoing, iterative study. Future research building upon this report should investigate:

1. **Multi-Agent Systems:** Evaluating the efficiency of using an ecosystem of narrow, specialized agents (e.g., one agent for physics coding, one for asset generation, and one adversarial agent solely for QA testing) rather than relying on a single monolithic model.
2. **Longitudinal Legal Tracking:** Tracking active legal challenges regarding the copyrightability of hybrid human-AI codebases to establish clearer guidelines for indie studios utilizing LLMs.
3. **Engine-Specific Benchmarking:** Expanding the experimental framework to evaluate AI model efficacy within C++ (Unreal Engine) and C# (Unity) environments, focusing heavily on the integration of 3D mathematics and shader programming.

# References

- [1] Yannakakis, G. N., & Togelius, J. (2018). *Artificial Intelligence and Games*. Springer.
- [2] Bonis, J. (2026). *Tokens per Joule: How to Quantify and Reduce the Energy Footprint of Clinical LLM Inference*. John Snow Labs.
- [3] DEV Community. (2025). *AI & PCG: Revolutionizing Game Development*.
- [4] CapTech University. (2025). *AI in Video Game Development: From Smarter NPCs to Procedural Worlds*.
- [5] Stepwise. (2025). *Top 5 Application Challenges of Large Language Models (LLMs)*.
- [6] AAAI. (2026). *TokenPowerBench: Benchmarking the Power Consumption of LLM Inference*.
- [7] Cruz, L. (2025). *Comparing the energy efficiency of different LLM inference runtimes*. SustainableSE.
- [8] ML.ENERGY. (2026). *Diagnosing Inference Energy Consumption with the ML.ENERGY Leaderboard v3.0*.
- [9] Muxup. (2026). *Per-query energy consumption of LLMs*.
- [10] Global Carbon Project. (2026). *Carbon Intensity of Global Compute Grids*.
- [11] US Copyright Office. (2025). *Copyright Registration Guidance: Works Containing Material Generated by Artificial Intelligence*.
- [12] World Economic Forum. (2026). *The Future of Jobs Report: AI and Software Engineering*.
- [13] IndieGames.com. (2025). *The AI Flood: Navigating the New Indie Game Market*.
- [14] HigherEd Tech Review. (2026). *Adapting Computer Science Curricula in the Age of Autonomous LLMs*.
- [15] Environmental Tech Coalition. (2025). *The Hidden Water Cost of Generative AI Servers*.
- [16] Alcott, B. (2005). *Jevons' paradox*. *Ecological Economics*, 54(1), 9-21. [Adapted for 2026 AI context].

**Authors of document:**

Adam Cherek 207250

Paweł Mańczak 188756

Mateusz Nowak 189420